

UNIVERSIDAD AUTONOMA DE SAN LUIS POTOSI

U.A.S.L.P.

FACULTAD DE INGENIERIA

AREA DE CIENCIAS DE LA COMPUTACION

5/5/26

Preguntados UASLP

Integrantes:

- Hernández Valencia Héctor Enrique
- Morales García Emmanuel Lucio.
- Nieto Galilea
- Suarez Rivera María Irazema

Carrera: Ingeniería en Sistemas Inteligentes.

Materia: Interfaces gráficas con aplicaciones.

Profesor: Núñez Varela Alberto Salvador

Fecha: 05/05/3026.

Semestre: 2025-2026/II

PRIMERA PARTE

Introducción

Este proyecto tiene como objetivo el desarrollo de un juego de preguntas y respuestas, utilizando el lenguaje de programación C# y una base de datos en MySQL. El proyecto se realizó en equipo, usando herramientas como Visual Studio y GitHub para el desarrollo en conjunto del proyecto.

El juego permite al usuario seleccionar una categoría y responder preguntas aleatorias que pueden ser de tipo texto, imagen o audio. Cada pregunta cuenta con varias opciones de respuesta y el juego indica si la respuesta seleccionada es correcta o incorrecta. Además, las preguntas y respuestas se obtienen directamente desde una base de datos, lo que permite que el sistema sea más dinámico y fácil de modificar.

Durante el desarrollo del proyecto se aplicó el uso de conocimiento de bases de datos, programación en C#, diseño de interfaces gráficas y trabajo en equipo. Se presentarán, las tecnologías utilizadas y los resultados obtenidos durante el desarrollo del proyecto.

Objetivo general

Desarrollar un juego de preguntas y respuestas, utilizando C# y una base de datos en MySQL, aplicando el trabajo en equipo y el uso de herramientas de desarrollo colaborativo como GitHub.

Objetivos específicos

- Diseñar una base de datos relacional que permita almacenar preguntas, respuestas y categorías.
- Desarrollar una interfaz gráfica en C# que permita al usuario interactuar fácilmente con el sistema.
- Implementar preguntas de diferentes tipos, como texto, imagen y audio.
- Permitir que el sistema obtenga las preguntas y respuestas desde la base de datos.
- Aplicar el uso de GitHub para el trabajo en equipo y el control de versiones del proyecto.

Descripción general del sistema

El sistema desarrollado es un juego de preguntas y respuestas, en el cual el usuario puede seleccionar una categoría y responder diferentes preguntas relacionadas con ese tema. El programa muestra una pregunta junto con cuatro opciones de respuesta, y el usuario debe seleccionar la opción que considere correcta.

El sistema cuenta con preguntas de diferentes tipos, como preguntas de texto, preguntas con imágenes y preguntas con audio. Todas las preguntas y respuestas se almacenan en la base de datos, lo que permite que el sistema pueda obtenerlas de forma directa y cambiar automáticamente cada vez que el usuario responde una pregunta.

Cuando el usuario selecciona una respuesta, el sistema verifica si es correcta o incorrecta y muestra un mensaje correspondiente. Después de responder, el programa carga una nueva pregunta de la misma categoría.

El proyecto fue desarrollado en equipo utilizando herramientas como Visual Studio y GitHub, lo que permitió dividir el trabajo entre los integrantes y trabajar de forma organizada durante el desarrollo del proyecto.

Distribución y descripción de tareas, módulos y actividades entre los integrantes

Creación del repositorio del proyecto en GitHub

Esta actividad consistió en la creación del repositorio donde se almacenó todo el proyecto, así como los recursos necesarios para su funcionamiento, como imágenes y audios. El repositorio permitió que se pudiera trabajar de forma colaborativa, subir sus avances y mantener organizado el desarrollo del proyecto.

Desarrollo: Trabajo en equipo

Diseño del menú principal del sistema

Esta actividad consistió en el desarrollo de la interfaz principal del sistema, en la cual el usuario puede seleccionar la categoría antes de comenzar a responder preguntas. Se diseñó una interfaz fácil de usar, permitiendo al usuario seleccionar entre las diferentes categorías disponibles en el juego. Además, se trabajó en la organización visual de los botones y en la estructura general del formulario principal.

Desarrollo: Héctor Enrique Hernández Valencia

Diseño y creación de la base de datos

Esta parte consistió en el diseño de la estructura de la base de datos que permite almacenar toda la información del sistema, como las preguntas, respuestas y categorías. Se definieron las tablas necesarias, los campos correspondientes y la relación entre ellas para que el sistema pudiera funcionar correctamente. Registrar todas las categorías que serían utilizadas dentro del sistema, con el objetivo de organizar las preguntas de forma adecuada. Las categorías permiten que el usuario pueda seleccionar el tema de su preferencia antes de comenzar el juego, y además facilitan la organización de la información dentro de la base de datos. También se realizaron pruebas para verificar que la base de datos funcionara correctamente antes de integrarla al programa.

Desarrollo: Emmanuel Lucio Morales García

Creación de preguntas y respuestas para la base de datos

En esta actividad, cada integrante del equipo desarrolló el contenido correspondiente a una o más categorías. Esto incluyó la redacción de las preguntas, la creación de las opciones de respuesta y la selección de la respuesta correcta. Además, cada integrante creó los comandos SQL necesarios para insertar esa información en la base de datos.

Desarrollo: Trabajo en equipo

Distribución de categorías entre los integrantes

Cada integrante del equipo se encargó de desarrollar el contenido de diferentes categorías dentro del sistema, tanto en preguntas de texto como en preguntas con imágenes y audio.

- Galilea Nieto: Categorías de UASLP y Redes Sociales
- María Irazema Suarez Rivera: Categorías de Cine y Tecnología.
- Emmanuel Lucio Morales García: Categorías de Videojuegos y Fútbol
- Héctor Enrique Hernández Valencia: Categoría de Cultura Pop

Integración de recursos (imágenes y audios)

Esta actividad consistió en la búsqueda y organización de los recursos necesarios para el funcionamiento del sistema, como imágenes y archivos de audio. Cada integrante buscó los recursos correspondientes a su categoría y los agregó al proyecto, asegurándose de que funcionaran correctamente dentro del sistema. También se organizó la información en carpetas para mantener el proyecto ordenado.

Desarrollo: Trabajo en equipo

Conexión entre el programa en C# y la base de datos

Esta parte consistió en establecer la conexión entre el sistema desarrollado en C# y la base de datos en MySQL, con el objetivo de que el programa pudiera obtener las preguntas de forma automática. Se realizaron pruebas para verificar que la información se cargara correctamente desde la base de datos al sistema.

Desarrollo: Trabajo en Equipo

Diseño de la interfaz para responder preguntas

Esta actividad consistió en el diseño de las pantallas donde el usuario responde las preguntas. Se trabajó en la organización visual de los elementos, como el texto de la pregunta, los controles de respuesta y el diseño general del formulario. El objetivo fue crear una interfaz clara, sencilla y fácil de usar para el usuario.

Desarrollo: Galilea Nieto.

Implementación de preguntas con imágenes

Esta etapa consistió en agregar la funcionalidad necesaria para que el sistema pudiera mostrar imágenes como opciones de respuesta. Se realizaron pruebas para verificar que las imágenes se cargaran correctamente desde la base de datos y que el sistema pudiera identificar la respuesta correcta.

Desarrollo: María Irazema Suarez Rivera

Implementación de preguntas con audio

En esta actividad se desarrolló la funcionalidad para que el sistema pudiera reproducir archivos de audio como parte de las preguntas. Se realizaron pruebas para verificar que el audio funcionara correctamente y que el sistema pudiera identificar la respuesta correcta después de reproducirlo.

Desarrollo: Galilea Nieto y María Irazema Suarez Rivera.

Registro de las partidas y sus respuestas

En esta parte del proyecto se realizó la conexión con la base de datos, para guardar las partidas en la tabla partidas, con su respectivo id, la cantidad de respuestas correctas, para almacenar cada partida.

Después de esto se relaciona con la tabla de respuestas_partida, donde se almacena un registro de cada partida, con las preguntas realizadas en esa partida, y guardar que respuestas tuvo correctas y cuales tuvo incorrectas, así finalizando la estructura de la bases de datos, almacenar toda la información para manejarla.

Desarrollo: Emmanuel Lucio Morales García.

Tecnologías utilizadas

Durante el desarrollo del sistema se utilizaron diferentes herramientas y tecnologías que permitieron crear la interfaz gráfica, almacenar la información y trabajar de forma colaborativa entre los integrantes del equipo.

Entorno de desarrollo (IDE)

Para el desarrollo del proyecto se utilizó Visual Studio, el cual permitió programar en el lenguaje C# y diseñar la interfaz gráfica mediante Windows Forms. Esta herramienta facilitó la creación de formularios, controles, etiquetas e imágenes necesarias para el funcionamiento del juego, además de permitir la conexión con la base de datos.

Lenguaje de programación

El sistema fue desarrollado utilizando el lenguaje de programación C#, el cual permitió crear la lógica del programa, mostrar las preguntas al usuario, validar las respuestas y conectar el sistema con la base de datos. Este lenguaje también permitió trabajar con imágenes y archivos de audio dentro del sistema.

Manejador de base de datos

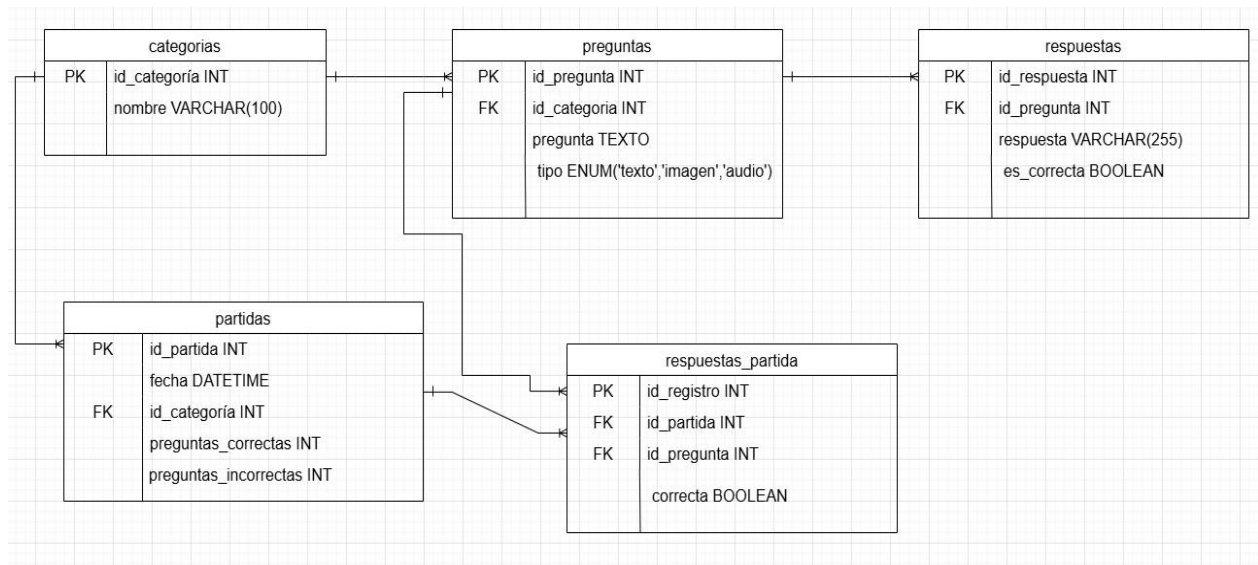
Para el almacenamiento de la información se utilizó MySQL, el cual permitió guardar las preguntas, respuestas y categorías de forma organizada. La base de datos fue diseñada utilizando tablas relacionadas entre sí, lo que permitió que el sistema pudiera obtener la información de forma dinámica.

Biblioteca gráfica

Para el diseño de la interfaz gráfica se utilizó Windows Forms, que permitió crear las pantallas del sistema, como el menú principal, la selección de categorías y las pantallas donde el usuario responde las preguntas. También permitió trabajar con los eventos y controles, imágenes y archivos de audio dentro del programa.

Modelo relacional de la base de datos

La base de datos está compuesta por cinco tablas principales: categorías, preguntas, respuestas, partidas y respuestas_partida. Estas tablas se encuentran relacionadas entre sí mediante claves primarias y claves foráneas, lo que permite organizar correctamente la información del sistema.



El modelo relacional de la base de datos fue diseñado con el objetivo de organizar de manera estructurada la información del proyecto. La tabla categorías permite clasificar las preguntas según su temática, mientras que la tabla preguntas almacena el contenido de cada pregunta junto con su tipo (texto, imagen o audio).

A su vez, cada pregunta se relaciona con la tabla respuestas, donde se guardan las opciones posibles y se identifica cuál es la correcta. Para registrar la actividad del usuario, se implementó la tabla partidas, la cual almacena la información de cada juego realizado, y la tabla respuestas_partida, que guarda el historial de respuestas durante cada partida. Todas las tablas se encuentran relacionadas mediante claves primarias y foráneas, lo que garantiza la integridad y consistencia de los datos dentro del sistema.

Listado de commits

Durante el desarrollo del proyecto se utilizó GitHub lo que permitió llevar un registro detallado de los cambios realizados por cada integrante del equipo. Cada miembro realizó commits conforme avanzaba en el desarrollo de su parte correspondiente, como la creación de preguntas, el diseño de interfaces, la conexión con la base de datos y la integración de recursos.

Los commits registrados incluyen información como el nombre del usuario, la fecha en que se realizó el cambio y una breve descripción de la modificación realizada, esto permitió mantener un control claro del avance del proyecto.

Usuarios – Alumnos:

HectorHacheve: Héctor Enrique Hernández Valencia.

Gali: Galilea Nieto

gali-galii: Galilea Nieto

irazema: María Irazema Suarez Rivera

itslucio37: Emmanuel Lucio Morales García

Emmanuel8151: Emmanuel Lucio Morales García

Algunos Commits fueron realizados por integrantes que para no generar problemas con el empalme del proyecto fueron subidos por otros miembros del equipo.

Usuario	Fecha	Descripción commit
irazema	24 de marzo	Partidas guardadas en base de datos
irazema	24 de marzo	Merge branch 'master' of https://github.com/HectorHacheve/PreguntadosUASLP
irazema	24 de marzo	Agregue pantalla final con opciones reiniciar/salir
Gali	24 de marzo	Conv. de labelPregunta a pictureBox
Gali	24 de marzo	Conv. de labels a pictureBox (num. de pregunta y categoría)

Gali	24 de marzo	PictureBox.SizeMode.Zoom se cambió a pictureBox.SizeMode.StretchImage
Gali	24 de marzo	Se ajustaron los botones de audio, cambio de color y texto
Gali	24 de marzo	Conv. de opciones de audio a pictureboxes
Gali	24 de marzo	Ajuste de op. de texto
Gali	24 de marzo	Conv. labels (op. texto) a pictureBox
Gali	24 de marzo	Se reemplazó botonConfirmar con Enter
Gali	24 de marzo	Se borró botonConfirmar
Gali	24 de marzo	Puntaje ahora cuenta correctos e incorrectos, se cambió a un pictureBox
Gali	24 de marzo	Cambio de nombre a func. AjustarPregunta
Gali	24 de marzo	Se arregló posición y tamaño de pregunta
Gali	24 de marzo	Se agregaron las imágenes para cat. Cultura pop en FormJuego
Gali	24 de marzo	Se arregló tamaño del texto
HectorHacheve	24 de marzo	Cultura pop sql
HectorHacheve	24 de marzo	AudiosPop
HectorHacheve	24 de marzo	Culturapop
HectorHacheve	23 de marzo	Añadir audios cultura pop
HectorHacheve	23 de marzo	Añadir imágenes cultura pop
Gali	23 de marzo	Se conectó el evento click de cat. Videojuegos y se borró duplicado
Gali	23 de marzo	Se elimina archivo duplicado de preguntas UASLP
Itslucio37	23 de marzo	Se agregan audios de la sección de videojuegos en la carpeta audios con A mayúscula
Itslucio37	23 de marzo	Se agregan audios de la sección de videojuegos a la carpeta audios con a minúscula

Itslucio37	23 de marzo	Se agregan imágenes de la sección de videojuegos a la carpeta imágenes con I mayúscula
Itslucio37	23 de marzo	Se agregan imágenes de la sección de videojuegos a la carpeta imágenes con i minúscula
Itslucio37	23 de marzo	Se agrega sección de videojuegos para la base de datos, preguntas y respuestas
Gali	23 de marzo	Merge remote-tracking branch 'origin/ira-avances'
Gali	23 de marzo	intento de acomodar pictureboxes en FormJuego
Gali	23 de marzo	Se agregaron imagenes y audios de cat. futbol a otra carpeta
Gali	23 de marzo	Ajustes en diseño de FormJuego
Gali	23 de marzo	Merge resuelto + agrego preguntas UASLP
Gali	23 de marzo	Ajustes en diseño de FormJuego
gali-galii	23 de marzo	preguntas y respuestas para cat. UASLP
Gali	23 de marzo	se cambió de lugar llamada a la siguiente pregunta a FormJuego_Load
Gali	23 de marzo	se corrigió directorio de imagenes y audios de cat. Redes Sociales y cat. UASLP
Gali	23 de marzo	se agregaron imagenes y audios de cat. Redes Sociales y cat. UASLP
gali-galii	23 de marzo	se agregaron las preguntas de la categorial UASLP
irazema	23 de marzo	Agregue preguntas de cine y tecnología que faltaban para que sean un total 12 preguntas por categoría
gali-galii	23 de marzo	Se actualizaron los nombres de las cat. UASLP y Redes Sociales. También se agregaron las preguntas de Redes sociales
itslucio37	23 de marzo	Se agregan las preguntas y respuestas para la base de datos de la sección de futbol
Itslucio37	23 de marzo	Se agregan audios de la sección de futbol
Itslucio37	23 de marzo	Se agregan imágenes para la selección de preguntas de futbol

irazema	22 de marzo	Agregue preguntas de tecnología con imágenes y audio a la base de datos
irazema	22 de marzo	Agregar audios para preguntas de tecnología
irazema	22 de marzo	Feat: agregar imágenes para preguntas de tecnología (Linux, ASIMO, disquete)
irazema	22 de marzo	Implemente preguntas de audio con WAV, botón confirmar y detener audio al cambiar pregunta
irazema	22 de marzo	Agrega los audios para las preguntas de cine.
irazema	22 de marzo	Implemente la visualización de imágenes en FormJuego en cine.
irazema	22 de marzo	Agrega imágenes para preguntas de cine
irazema	22 de marzo	Evitar preguntas repetidas y mostrar contador
irazema	22 de marzo	Combinar lógica de juego con imágenes de categoría
irazema	22 de marzo	Hazaña: implementar FormJuego con carga de preguntas
Gali	22 de marzo	Se agregó UASLP y Redes Sociales a FormJuego junto con sus imágenes
Gali	22 de marzo	Se actualizó Resources y func. para agregar imágenes según la categoría
irazema	22 de marzo	Fusionar la rama 'master' de https://github.com/HectorHacheve/PreguntadosUASLP en ira-avances
Gali	22 de marzo	Se agregó func. para mostrar botones de respuestas
Gali	22 de marzo	Cambios de nombre en el diseño de FormJuego, ajustes y organización en Designer.cs
irazema	22 de marzo	Fusionar la rama 'master' de https://github.com/HectorHacheve/PreguntadosUASLP en ira-avances
Gali	22 de marzo	Se agregaron los botones para respuestas de txt, img y audio al diseño de FormJuego.
irazema	21 de marzo	Agregué preguntas y respuestas normales
Gali	21 de marzo	Ajustes en el diseño de FormJuego.
Gali	20 de marzo	Se agregó diseño base de FormJuego con etiquetas y envío de categoría
Gali	20 de marzo	Se agregó FormJuego y conexión de cat. UASLP

Gali	20 de marzo	Se agregaron imágenes a Recursos para cat. UASLP y cat. Redes Sociales
Gali	20 de marzo	Arreglo de error en recursos y ajuste pequeños en la interfaz principal
Gali	18 de marzo	Ajustes pequeños en la página principal
Itslucio37	17 de marzo	Estructura de la base de datos
Emmanuel8151	17 de marzo	Pruebas de implementación base de datos prueba
Emmanuel8151	16 de marzo	Cambios Futbol
Emmanuel8151	16 de marzo	Fútbol
Emmanuel8151	16 de marzo	Formulario de fútbol
Emmanuel8151	16 de marzo	Ventana de futbol, llegada las chivas de Guadalajara DAVOOOO
Emmanuel8151	16 de marzo	Pantalla de futbol, arriba las chivas de Guadalajara
HectorHacheve	16 de marzo	Diseño inicial interfaz completa
HectorHacheve	16 de marzo	Agregar archivos de proyecto.
HectorHacheve	16 de marzo	Agregar .gitattributes y .gitignore.

Se compromete el 24 de marzo de 2026.

Partidas guardadas en base de datos irazema comprometido hace 3 minutos	62b25b1	<>
Merge branch 'master' of https://github.com/HectorHacheve/PreguntadosUASLP irazema comprometido hace 30 minutos	46786b3	<>
Agregue pantalla final con opciones reiniciar/salir irazema comprometido hace 31 minutos	2534ab4	<>
conv. de labelPregunta a pictureBox Gali comprometido hace 43 minutos	72da49b	<>
conv. de labels a pictureBox (num. de pregunta y categoría) Gali comprometido hace 54 minutos	b6da3f5	<>
pictureBox.SizeMode.Zoom se cambió a pictureBox.SizeMode.StretchImage Gali comprometido hace 2 horas	6ea9e68	<>
se ajustaron los botones de audio, cambio de color y texto Gali comprometido hace 2 horas	2b8279a	<>
conv. de opciones de audio a pictureBoxes Gali comprometido hace 2 horas	b936036	<>

ajuste de op. de texto  Gali comprometido hace 3 horas	f771fe		
conv. labels (op. texto) a pictureBox  Gali comprometido hace 3 horas	5849395		
se reemplazó botonConfirmar con Enter  Gali comprometido hace 4 horas	25fbd2e		
se borró botonConfirmar  Gali comprometido hace 4 horas	987a161		
puntaje ahora cuenta correctos e incorrectos. se cambió a un pictureBox  Gali comprometido hace 9 horas	78b884f		
cambio de nombre a func. AjustarPregunta  Gali comprometido hace 9 horas	188beb6		
se arregló posición y tamaño de pregunta  Gali comprometido hace 9 horas	22d55d4		
se agregaron las imágenes para cat. Cultura pop en FormJuego  Gali comprometido hace 10 horas	ac4944e		
se arregló tamaño del texto  Gali comprometido hace 10 horas	bf53bb9		

se arregló tamaño del texto  Gali comprometido hace 10 horas	bf53bb9		
cultura pop sql  Héctor Hachev autor hace 11 horas	Verified 9f9db10		
se agregaron imágenes a las categorías  Gali comprometido hace 11 horas	0f352b9		
se agregaron imágenes para las categorías a Recursos  Gali comprometido hace 11 horas	f660c67		
audiosPop  Héctor Hachev autor hace 12 horas	Verified c7bdcf7		
culturapop  Héctor Hachev autor hace 12 horas	Verified 7873c4b		

 Se compromete el 23 de marzo de 2026.

Añadir audios cultura pop  Héctor Hachev autor hace 35 minutos	Verified 519fab0		
Añadir imágenes cultura pop  Héctor Hachev autor hace 35 minutos	Verified 8149da8		
se conectó el evento click de cat. Videojuegos y se borró duplicado  Gali comprometido hace 1 hora	bc34d04		
se elimina archivo duplicado de preguntas UASLP  Gali comprometido hace 1 hora	63f3310		
Agregar archivos mediante carga   itslucio37-dev autor hace 6 horas	Verified 9621b26		
Agregar archivos mediante carga   itslucio37-dev autor hace 6 horas	Verified 61a9b0c		
Agregar archivos mediante carga   itslucio37-dev autor hace 6 horas	Verified 78 muertos 2		
Agregar archivos mediante carga   itslucio37-dev autor hace 6 horas	Verified e6c0942		

Agregar archivos mediante carga   itslucio37-dev autor hace 6 horas	Verified 022297b  
Fusionar la rama de seguimiento remoto 'origin/ira-avances'  Gali comprometido hace 12 horas	ccd733a  
intento de acomodar cuadros de fotos en FormJuego  Gali comprometido hace 12 horas	ff317cb  
se agregaron imágenes y audios de cat. fútbol en otra carpeta  Gali comprometido hace 13 horas	8526ff7  
ajustes en diseño de FormJuego  Gali comprometido hace 13 horas	1abd56c  
fusionar resuelto + agregado preguntas UASLP  Gali comprometido hace 13 horas	bb92ee9  
ajustes en diseño de FormJuego  Gali comprometido hace 13 horas	37becde  
preguntas y respuestas para cat. UASLP  gali-galii autor hace 14 horas	Verified 409a724  
se cambió de lugar llamada a la siguiente pregunta a FormJuego_Load  Gali comprometido hace 14 horas	bb1264a  

se corrigió directorio de imágenes y audios de cat. Redes Sociales y cat. UASLP  Gali comprometido hace 14 horas	d601718  
se agregaron imágenes y audios de cat. Redes Sociales y cat. UASLP  Gali comprometido hace 14 horas	54493b2  
se agregaron las preguntas de la categoria UASLP  gali-galii autor hace 15 horas	Verified b6fe19d  
Agrega preguntas de cine y tecnología que faltaban para que sean un total de 12 preguntas por categoría.  irazema comprometido hace 18 horas	58da643  
agregar preguntas de la cat. redes sociales   gali-galii autor ayer	Verified 88f768a  
Agregar archivos mediante carga   itslucio37-dev autor ayer	Verified 3387193  
Agregar archivos mediante carga   itslucio37-dev autor ayer	Verified e68804c  
Agregar archivos mediante carga   itslucio37-dev autor ayer	Verified 9c254cc  

 Se compromete el 22 de marzo de 2026.

Agregar preguntas de tecnología con imagenes y audio a la base de datos  irazema comprometido ayer	1a513d3  
agregar audios para preguntas de tecnologia  irazema comprometido ayer	57e19c3  
feat: agregar imagenes para preguntas de tecnologia (Linux, ASIMO, disquete)  irazema comprometido ayer	15758d0  
implemente preguntas de audio con WAV, boton confirmar y detener audio al cambiar pregunta  irazema comprometido ayer	28974d6  
Agrega los audios para las preguntas de cine.  irazema comprometido ayer	2a6942d  
Implemente la visualización de imágenes en FormJuego en cine.  irazema comprometido ayer	e878849  
Agrega imagenes para preguntas de cine  irazema comprometido ayer	3d3f217  
evitar preguntas repetidas y mostrar contador  irazema comprometido anteaayer	1c604ff  
combinar lógica de juego con imágenes de categoría  irazema comprometido anteaayer	d0c343d  

Se compromete el 22 de marzo de 2026.

se agregó UASLP y Redes Sociales a FormJuego junto con sus imágenes Gali comprometido anteyayer	6280ab0		<>
se actualizó Resources y func. para agregar imágenes según la categoría Gali comprometido anteyayer	f65fb22		<>
Fusionar la rama 'master' de https://github.com/HectorHacheve/PreguntadosUASLP en ira-avances irazema comprometido anteyayer	0f0fe90		<>
se agregó func. para mostrar botones de respuestas Gali comprometido anteyayer	ed943db		<>
cambios de nombre en el diseño de FormJuego, ajustes y organización en Designer.cs Gali comprometido anteyayer	377faeb1		<>
Fusionar la rama 'master' de https://github.com/HectorHacheve/PreguntadosUASLP en ira-avances irazema comprometido anteyayer	ba58eee		<>
Se agregaron los botones para respuestas de txt, img y audio al diseño de FormJuego. Gali comprometido anteyayer	97db6cb		<>

Se compromete el 21 de marzo de 2026.

Agregué preguntas y respuestas normales irazema comprometido anteyayer	a44e0fb		<>
Ajustes en el diseño de FormJuego. Gali comprometido anteyayer	47f6f63		<>

Se compromete el 20 de marzo de 2026.

se agregó diseño base de FormJuego con etiquetas y envío de categoría Gali comprometido hace 3 días	d9de1df		<>
se agregó FormJuego y conexión de cat. UASLP Gali comprometido hace 3 días	6120468		<>
se agregaron imágenes a Recursos para cat. UASLP y cat. Redes Sociales Gali comprometido hace 3 días	6d59633		<>
arreglo de error en recursos y ajuste pequeños en la interfaz principal Gali comprometido hace 4 días	1d9a160		<>

Se compromete el 18 de marzo de 2026.

ajustes pequeños en la página principal Gali comprometido hace 5 días	8674f16		<>
---	---------	---	----

Se compromete el 17 de marzo de 2026.

Agregar archivos mediante carga   itslucio37-dev autor la semana pasada	 37fe3d1		<>
Pruebas de implementación base de datos prueba  Emmanuel8151 comprometido la semana pasada	b793ac1		<>

Se compromete el 16 de marzo de 2026.

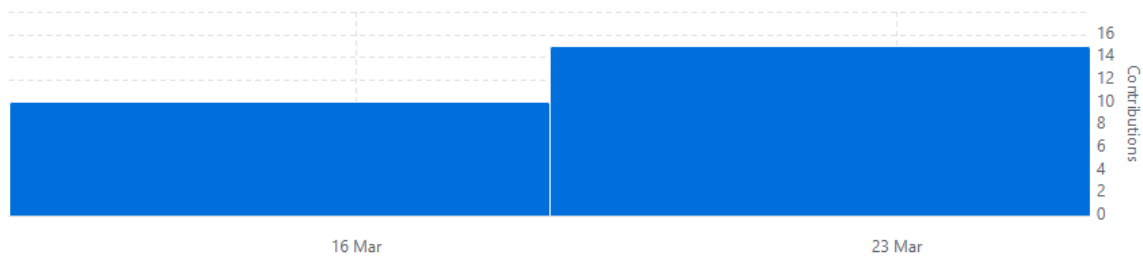
Cambios Futbol Emmanuel8151 comprometido la semana pasada	8ed5516		
Fútbol Emmanuel8151 comprometido la semana pasada	274fac9		
Formulario de fútbol Emmanuel8151 comprometido la semana pasada	712b6bb		
Ventana de futbol, llegada las chivas de Guadalajara DAVOOOO Emmanuel8151 comprometido la semana pasada	5366ef9		
Pantalla de futbol, arriba las chivas de guadalajara Emmanuel8151 comprometido la semana pasada	6062de1		
Diseño inicial interfaz completa Héctor Hachev comprometido la semana pasada	c6beb6a		
Agregar archivos de proyecto. Héctor Hachev comprometido la semana pasada	88a72b4		
Agregar .gitattributes y .gitignore. Héctor Hachev comprometido la semana pasada	24ef3d3		

Mediante la sección Insights del repositorio de GitHub se pudieron obtener las estadísticas generales del proyecto, donde se muestra el número de commits, así como la frecuencia de trabajo durante el desarrollo del sistema.

Commits over time

Commits over time

Weekly from 14 mar 2026 to 21 mar 2026



Commits



Code frequency



Problemas encontrados al usar Git y GitHub

Durante el desarrollo del proyecto se presentaron varios problemas relacionados con el uso de GitHub. Al tratarse de un trabajo en equipo, fue necesario organizar los cambios realizados por cada integrante, lo que generaba problemas dentro del repositorio.

Uno de los principales problemas fue que se realizaban cambios en el proyecto y se subían al repositorio sin haber actualizado antes. Esto hacía que los cambios no se integraran correctamente o generaba problemas al subir nuevas modificaciones.

Otro problema ocurrió con la estructura del proyecto, especialmente cuando se agregaron carpetas nuevas como la carpeta de imágenes y audios. En algunos casos, estas carpetas sí aparecían en el repositorio, pero no se mostraban automáticamente dentro del proyecto en Visual Studio, lo que provocaba errores al ejecutar el programa, ya que el sistema no encontraba los archivos necesarios.

También se presentaron problemas al trabajar con archivos multimedia, debido a que las rutas de las imágenes y los audios no coincidían en todos los equipos. Esto ocasionaba que, aunque el código funcionara correctamente, las imágenes no se mostraran o los audios no se reprodujeran.

Otro problema era al modificar mismos archivos al mismo tiempo, lo que generó varios errores, a pesar de estos problemas, el equipo logró adaptarse al uso de GitHub, realizando los commits y los cambios adecuados al proyecto y al repositorio.

Problemas encontrados durante el desarrollo del proyecto

Durante el desarrollo del proyecto se presentaron diversos problemas, uno de los primeros problemas fue la conexión entre el programa y la base de datos, ya que al inicio el programa no obtenía las preguntas almacenadas.

Otro problema fue al trabajar con preguntas que incluían imágenes y audios. En varias ocasiones el programa mostraba únicamente los controles o el contorno de la imagen, pero no la imagen, debido a que las rutas de los archivos no coincidían con la estructura del proyecto.

Además, se presentaron problemas con la organización del trabajo, sobre todo al inicio del proyecto, ya que no estaba completamente claro cómo se dividirían todas las actividades.

Conclusión

En este proyecto se logró desarrollar un juego de preguntas, utilizando programación, base de datos, manejo de GitHub y el trabajo en equipo. Durante el desarrollo se implementaron preguntas de texto, imagen y audio, además de la conexión con la base de datos para obtener la información.

También se presentaron algunos problemas, con la organización de las carpetas de imágenes y audios, y la sincronización del proyecto en GitHub. Sin embargo, se lograron solucionar los problemas hasta que el sistema funcionó correctamente.

En general, el proyecto ayudó a comprender mejor cómo trabajar en equipo, cómo organizar un proyecto en Visual Studio utilizando una base de datos y cómo utilizar GitHub para compartir y actualizar avances.

SEGUNDA PARTE MULTIJIUGADOR

Introducción

Esta etapa del proyecto representa un cambio significativo en la arquitectura del sistema, la fase inicial se centró en la gestión local de datos y la lógica fundamental del juego, este nuevo desarrollo actualiza la aplicación hacia un modelo de computación distribuida mediante la implementación de una interfaz de programación de aplicaciones API.

El objetivo principal de esta actualización es independizar la interfaz de usuario de la persistencia de datos. Para ello, se ha configurado una computadora que actúa como servidor, encargada de gestionar las consultas a la base de datos MySQL. Las terminales de los jugadores ahora operan como clientes que, a través de una red local, usan los recursos como preguntas, imágenes y audios de manera simultánea.

Este cambio permite que varios usuarios conectados a la misma red participen en una partida sincronizada, donde la API mediante el servidor y con ayuda del socket actúa como el único mediador de la información.

Objetivo General

Actualizar el sistema de juegos de preguntas y respuestas mediante una arquitectura Cliente-Servidor, utilizando el lenguaje C# usando una API con una base de datos MySQL con protocolos de comunicación por red local.

Objetivos Específicos

- **Diseño de Persistencia:** Mantener y optimizar una base de datos relacional para el almacenamiento estructurado de reactivos, opciones multimodales y el historial de partidas.
- **Desarrollo de API:** Implementar una interfaz de programación de aplicaciones que gestione las peticiones de los clientes, centralice la lógica y sirva como único punto de acceso a la base de datos.
- **Comunicación en Tiempo Real (Sockets):** Integrar el uso de Sockets para permitir la comunicación bidireccional entre el servidor y los clientes, asegurando que todos los jugadores visualicen la misma partida simultáneamente.
- **Gestión de Sesiones e Interfaz de Espera:** Diseñar interfaces gráficas dinámicas para la creación de salas de espera, donde se gestione el ingreso de jugadores y la sincronización previa al inicio de la competencia.
- **Multimedia y Dinamismo:** Consolidar el sistema de carga de recursos para soportar preguntas de tipo texto, imagen y audio, enviadas desde el servidor hacia las terminales cliente de forma eficiente.

- Colaboración Técnica: Documentar y gestionar el ciclo de vida del software utilizando GitHub, permitiendo la integración de las nuevas funcionalidades de red y el control de versiones de forma organizada.

Descripción general del sistema

El sistema cambió de una aplicación local a una plataforma de competencia distribuida basada en una arquitectura Cliente-Servidor. En esta nueva configuración, el núcleo del juego reside en un servidor central que ejecuta una API, la cual actúa como el único intermediario encargado de realizar las consultas y transacciones con la base de datos MySQL.

El flujo de interacción del sistema se divide en varias etapas principales.

Primero conexión y lobby, los jugadores se conectan al servidor utilizando la dirección IP de la computadora central. Al ingresar, son dirigidos a una interfaz de Lobby, donde, gracias al uso de Sockets, se puede visualizar en tiempo real a todos los participantes que se han unido a la sesión antes de iniciar la competencia.

Sincronización de partida, una vez que se inicia el juego y se selecciona la categoría, el servidor gestiona una partida simultánea. Esto garantiza que todos los clientes reciban exactamente la secuencia de preguntas y opciones, texto, imagen o audio, las cuales son distribuidas por la API desde el servidor centralizado.

Cada usuario interactúa con su propia interfaz para responder a los reactivos enviados por el servidor. Al finalizar la ronda de preguntas, el sistema procesa los datos y despliega una pantalla de resultados finales, donde se detallan de forma individual los aciertos y errores de cada jugador.

El desarrollo se mantuvo bajo un esquema de trabajo colaborativo en GitHub, permitiendo la integración de la lógica de red y la sincronización de interfaces de forma organizada entre los miembros del equipo.

Implementación de la API y Comunicación en Tiempo Real

Para la gestión de los datos y la sincronización de la partida, se desarrolló un servidor utilizando API con Python el cual actúa como el motor central del proyecto. La API REST para el manejo de datos y el LobbyManager para la gestión de jugadores mediante WebSockets.

Para la gestión de la base de datos se implementó una encargada de la conexión con MySQL. Teniendo la capacidad para manejar conexiones simultáneas, incluye un mecanismo de "ping" que verifica si la conexión sigue activa antes de cada consulta, reconectando automáticamente si es necesario, utiliza métodos específicos para consultas de lectura y escritura, integrando manejos de errores y casos de fallos.

La API expone diversas rutas que los clientes en C# consumen durante el juego, como Endpoints para obtener el listado de categorías y extraer preguntas aleatorias, permitiendo excluir aquellas que ya han sido respondidas en la sesión actual. Funciones para registrar una nueva partida en la base de datos, guardar cada respuesta individual y actualizar el puntaje final de los jugadores.

El componente que permite la interactividad en tiempo real sin necesidad de que el cliente esté refrescando la información manualmente, gestionando las conexiones activas y notifica automáticamente a todos los clientes cuando un nuevo jugador se une o se retira de la sala, permite que el "Host" inicie la partida para todos simultáneamente, enviando la categoría seleccionada a través de un mensaje.

Distribución y descripción de tareas, módulos y actividades entre los integrantes

Diseño e implementación de la api

Se desarrolló una interfaz de programación de aplicaciones API para centralizar la lógica de acceso a datos. Se definieron endpoints para la extracción de categorías, preguntas Trabajo en equipo.

Gestión de Persistencia y Pool de Conexiones: Se diseñó el módulo DatabaseManager para administrar la comunicación con MySQL. Este módulo incluye un sistema para manejar conexiones simultáneas y un mecanismo de reconexión automática que garantiza la estabilidad del servicio ante caídas de red.

Desarrollo: Héctor Enrique Hernández Valencia

Arquitectura de Sockets para el Servidor: Se implementó el componente LobbyManager, el cual utiliza el protocolo WebSockets para gestionar la comunicación bidireccional y en tiempo real. Este módulo se encarga de realizar el broadcast de mensajes para actualizar la lista de jugadores y sincronizar el inicio de la partida para todos los clientes conectados.

Desarrollo: Héctor Enrique Hernández Valencia

Programación del Socket Cliente

Se desarrolló la lógica necesaria en C# para establecer una conexión persistente con el servidor mediante la dirección IP. El cliente es capaz de interpretar mensajes en formato JSON para actualizar la interfaz de usuario dinámicamente según las instrucciones del servidor.

Desarrollo: Emmanuel Lucio Morales García.

Módulo de Sincronización de Juego

Se adaptó el formulario principal de juego para que, en lugar de consultar la base de datos localmente, realice peticiones asíncronas a la API para obtener los reactivos y registre cada respuesta de forma remota.

Desarrollo: Trabajo en equipo.

Interfaz de Lobby

Se diseñó una nueva pantalla que permite a los usuarios visualizar en tiempo real quiénes están conectados antes de que el "Host" inicie la competencia.

Desarrollo: Galilea Nieto.

Visualización de Resultados

Se implementó una lógica de cierre de partida que, al detectar que todos los jugadores han terminado, solicita al servidor el resultado de cada jugador para mostrarlo simultáneamente en todas las terminales.

Desarrollo: María Irazema Suarez Rivera.

Configuración de Red Local

Se realizaron pruebas de conectividad para asegurar que los clientes pudieran alcanzar el servidor mediante el protocolo HTTP y WebSockets a través de la red de la Universidad.

Desarrollo: Trabajo en equipo

Control de Versiones en GitHub

Se gestionó el repositorio del proyecto para integrar el código de Python del Servidor y C# del Cliente, permitiendo un desarrollo paralelo y organizado entre los integrantes del equipo.

Desarrollo: Trabajo en equipo.

Tecnologías utilizadas

En esta segunda fase, las herramientas se expandieron para soportar una arquitectura distribuida, permitiendo la comunicación entre múltiples clientes y un servidor centralizado.

Lenguaje de programación C#

Se mantuvo como el lenguaje principal para la lógica del cliente, encargado de la gestión de eventos de la interfaz, la reproducción de recursos multimedia y la comunicación asíncrona con la API.

Entorno de Desarrollo (Visual Studio)

Utilizado para la codificación, depuración y diseño de los formularios de Windows Forms.

Windows Forms

Biblioteca utilizada para el diseño de las interfaces gráficas, incluyendo el nuevo lobby multijugador y la pantalla de resultados compartidos.

Lenguaje de programación Python

Se eligió para el desarrollo del servidor debido a su eficiencia en el manejo de peticiones concurrentes y su robustez para servicios web.

Tecnología utilizada para construir la API que sirve los datos del juego, por su alta velocidad y su soporte nativo para operaciones asíncronas.

Protocolo WebSocket

Implementado para establecer una comunicación bidireccional y persistente entre el servidor y los clientes, permitiendo actualizaciones en tiempo real como la lista de jugadores en el lobby sin necesidad de peticiones constantes.

Arquitectura API REST

Utilizada para la transferencia de datos estructurados entre la base de datos y los clientes mediante el protocolo HTTP.

Formato JSON

Estándar utilizado para el intercambio de información entre el servidor de Python y el cliente de C#, asegurando que ambos sistemas interpreten correctamente las preguntas y resultados.

Manejador de Base de Datos (MySQL)

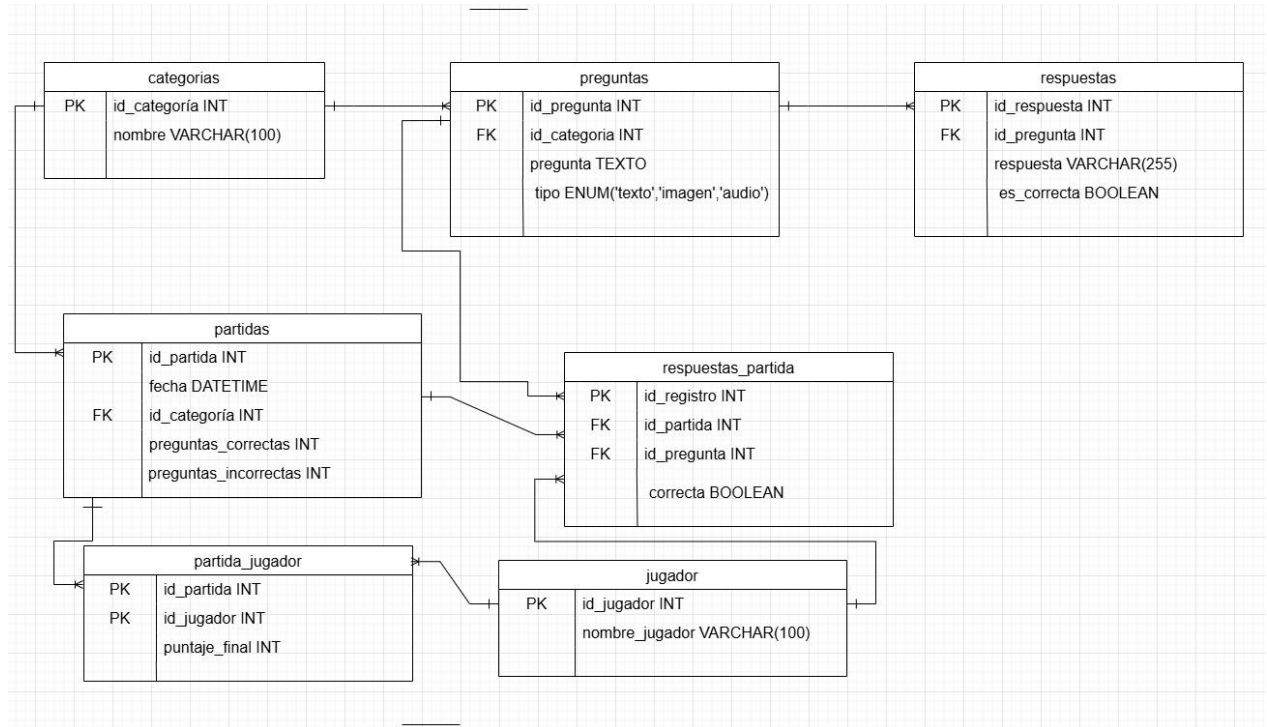
Sigue siendo el motor de almacenamiento principal donde residen las tablas de preguntas, respuestas, categorías y el historial de partidas.

GitHub

Plataforma esencial para el control de versiones, permitiendo integrar el código del servidor con el del cliente de manera fluida entre todos los integrantes del equipo.

Modelo relacional de la base de datos

La base de datos tablas principales, estas tablas se encuentran relacionadas entre sí mediante claves primarias y claves foráneas, lo que permite organizar correctamente la información del sistema.



Listado de commits

Durante el desarrollo del proyecto se utilizó GitHub lo que permitió llevar un registro detallado de los cambios realizados por cada integrante del equipo. Cada miembro realizó commits conforme avanzaba en el desarrollo de su parte correspondiente, como la creación de preguntas, el diseño de interfaces, la conexión con la base de datos y la integración de recursos.

Los commits registrados incluyen información como el nombre del usuario, la fecha en que se realizó el cambio y una breve descripción de la modificación realizada, esto permitió mantener un control claro del avance del proyecto.

Usuarios – Alumnos:

HectorHacheve: Héctor Enrique Hernández Valencia.

Gali: Galilea Nieto

gali-galii: Galilea Nieto

irazema: María Irazema Suarez Rivera

itslucio37: Emmanuel Lucio Morales García

Emmanuel8151: Emmanuel Lucio Morales García

Algunos Commits fueron realizados por integrantes que para no generar problemas con el empalme del proyecto fueron subidos por otros miembros del equipo.

Usuario	Fecha	Descripción commit
ItsLucio37	05 de mayo	Subir reporte Final
ItsLucio37	05 de mayo	Agregar Archivos Mediante Cargas
Irazema	05 de mayo	Modifique el formulario final para dar resultados en multijugador
HectorHacheve	04 de mayo	API
HectorHacheve	04 de mayo	Si
HectorHacheve	04 de mayo	No se
Gali	04 de mayo	Ingresa y espacio para FormFinal.cs lleva a FormLobby.cs y cierra el programa
Gali	03 de mayo	Marcadores de posición para API y Socket en Form1, FormJuego y FormLobby + ajustes para registrar jugador
Gali	03 de mayo	FormLobby.cs funcionando, falta agregar API y Socket
Gali	03 de mayo	Detalles al diseño final de FormLobby + celebrar.png
Gali	03 de mayo	Imagen decorativa agregada para FormLobby.png
Gali	02 de mayo	Diseño base para FormLobby + se agregaron titulo.png y logo.png a resources
Gali	02 de mayo	Fusionar de formjuego.cs
Gali	02 de mayo	Creación de FormLobby

Se compromete el 3 de mayo de 2026.

marcadores de posición para API y Socket en Form1, FormJuego y FormLobby + ajustes para registrar jugador	Gali comprometido anteayer	cdf9ad5 <>
FormLobby.cs funcionando, falta agregar API y Socket	Gali comprometido anteayer	ed60537 <>
detalles al diseño final de FormLobby + celebrar.png	Gali comprometido anteayer	cd1af81 <>
Imagen decorativa agregada para FormLobby.png	Gali comprometido anteayer	f435cf5 <>

Se compromete el 2 de mayo de 2026.

diseño base para FormLobby + se agregaron titulo.png y logo.png a resources	Gali comprometido hace 3 días	5948eca <>
fusionar de formjuego.cs	Gali comprometido hace 3 días	1736b23 <>
creación de FormLobby	Gali comprometido hace 3 días	563fb1e <>

Se compromete el 5 de mayo de 2026.

Agregar archivos mediante carga	Modelo Relacional Modificado itslucio37-dev autor hace 10 minutos	Verified dfe2cd0 <>
Modifique el formulario final para dar resultados en multijugador	irazema comprometido hace 15 horas	60ddc89 <>

Se compromete el 4 de mayo de 2026.

¿API?	Héctor Hachev comprometido ayer	952aa55 <>
si	Héctor Hachev comprometido ayer	61918ce <>
no sé	Héctor Hachev comprometido ayer	16dda4e <>
ingrese y espacio para FormFinal.cs lleva a FormLobby.cs y cierra el programa	Gali comprometido ayer	561220a <>

Se comoromete el 3 de mavo de 2026.

Commits over time



Commits

Compromisos

Número de confirmaciones por semana



Code frequency

Frecuencia del código

Adiciones y eliminaciones por semana



Problemas encontrados al usar GitHub

Durante esta segunda fase, la complejidad del repositorio aumentó al integrar dos lenguajes de programación C# y Python en un mismo proyecto, lo que derivó en los siguientes retos. Debido a que varios integrantes modificaban simultáneamente el archivo de la lógica del servidor, para añadir nuevos endpoints o ajustar el LobbyManager, se generaron conflictos frecuentes que requerían una revisión manual línea por línea para no perder funcionalidades.

Un problema recurrente fue la sincronización de las bibliotecas de Python. Mientras un integrante añadía cosas, otro necesitaba mysql-connector-python, lo que causaba que el servidor no ejecutara en todas las terminales hasta que se estandarizaron los archivos de requerimientos.

Al igual que en la fase anterior, las rutas de imágenes y audios causaron conflictos. Se tuvo que implementar una lógica de rutas relativas basadas en el directorio de ejecución de la API para que el servidor pudiera servir los archivos correctamente a todos los clientes, independientemente de la carpeta local de cada integrante.

Problemas encontrados durante el desarrollo del proyecto

El paso de una aplicación local a un sistema distribuido presentó desafíos técnicos que pusieron a prueba la estabilidad del software.

Inicialmente, se presentaron pérdidas de comunicación cuando un jugador cerraba la aplicación de forma inesperada. Se tuvo que robustecer el método disconnect en el servidor para evitar problemas

Durante las pruebas multijugador, el envío simultáneo de resultados al finalizar la partida saturaba brevemente el servidor. Esto se resolvió optimizando el LobbyManager para recolectar las puntuaciones en un diccionario de resultados y emitir un único mensaje de ranking cuando la cuenta de jugadores terminados coincidiera con los conectados.

Se detectaron errores de "conexión perdida" cuando la API pasaba mucho tiempo inactiva. Para solucionar esto, se implementó el métodos para reconectar si es necesario en el, el cual realiza un "ping" a MySQL antes de cada operación para asegurar que el enlace esté activo.

Uno de los obstáculos más complicados fue el bloqueo de puertos por los firewalls de los equipos. Fue necesario configurar manualmente las excepciones para los puertos y el asegurar que todos los clientes apuntaran correctamente a la dirección IP dinámica del servidor dentro de la red de la universidad.

Uno de los retos más complejos fue la reestructuración completa del código en C#. Originalmente, el juego realizaba peticiones directas a la base de datos mediante comandos SQL integrados en los formularios. Adaptar este flujo para que funcionara exclusivamente mediante la API requirió eliminar todas las dependencias de MySqlCommand en el cliente y sustituirlas por una arquitectura basada en HttpClient. Esto implicó cambiar las llamadas a funciones implementar una lógica de para evitar que la interfaz del juego se congelara mientras esperaba la respuesta del servidor a través de la red, así como la creación de objetos de transferencia de datos.

Conclusión

El desarrollo de la segunda fase del juego de preguntas y respuestas ha representado un desafío en la arquitectura del sistema, logrando la transición exitosa de una aplicación local hacia un entorno distribuido. A través de la implementación de una API y la integración de Sockets, se logró eliminar la dependencia directa de los clientes con la base de datos, centralizando la lógica en un servidor capaz de gestionar múltiples conexiones simultáneas en tiempo real.

Se trabajó la comunicación entre procesos mediante protocolos y Sockets, asegurando la sincronización de una misma partida para todos los usuarios en una red local.

La implementación de en C# y Python evitando bloqueos en la interfaz gráfica durante el intercambio de datos multimedia, aunque se presentaron muchos problemas en los archivos relacionados al audio.

El uso de GitHub fue vital para coordinar la integración de distintos lenguajes y módulos, superando retos técnicos mediante un control de versiones organizado.

En conclusión, el proyecto fue un desafío y reto muy complicado desde diseño de bases de datos, programación, API, sockets, sincronización, pero gracias al trabajo colaborativo de cada uno de los integrantes se lograron resultados satisfactorios y un buen aprendizaje.